UNIVERSIDAD DEL VALLE

EXAMEN PRÁCTICO INDIVIDUAL - 2DO MOMENTO

Superstore Sales Dataset (Sample Superstore)

Estudiante:	Ronald Joel Saavedra Vargas
Código / Email:	svr0035567@est.univalle.edu
Asignatura:	Tecnologías Emergentes I
Repositorio GitHub Oficial:	https://github.com/RJoel158/tem1_2mom_saavedra-ronald

1. OBJETIVO

Desarrollar un proyecto individual de procesamiento y análisis de datos utilizando Apache Hive, aplicando técnicas de modelado, ingesta, transformación, consultas analíticas avanzadas, optimización y generación de información para la toma de decisiones.
El estudiante deberá demostrar dominio técnico de HiveQL y capacidad para interpretar los resultados obtenidos, documentando íntegramente su trabajo mediante un repositorio individual en GitHub.

2. DATASET

Descargar el conjunto de datos desde:
https://www.kaggle.com/datasets/vivek468/superstore-dataset-final
Se deberá verificar la estructura original del archivo CSV, identificar sus campos, tipos de datos, formatos y posibles inconsistencias antes de realizar la ingesta.

3. DESARROLLO DEL PROYECTO

BLOQUE I. INGENIERÍA Y MODELADO DE DATOS

Pregunta 1. Diseño e implementación del modelo de datos

Construya el modelo de almacenamiento en Apache Hive para el dataset proporcionado.
Actividades:

Identificar todos los campos del dataset.

Row ID => Unique ID for each row.
Order ID => Unique Order ID for each Customer.
Order Date => Order Date of the product.
Ship Date => Shipping Date of the Product.
Ship Mode => Shipping Mode specified by the Customer.
Customer ID => Unique ID to identify each Customer.
Customer Name => Name of the Customer.
Segment => The segment where the Customer belongs.
Country => Country of residence of the Customer.
City => City of residence of the Customer.
State => State of residence of the Customer.
Postal Code => Postal Code of every Customer.
Region => Region where the Customer belong.
Product ID => Unique ID of the Product.
Category => Category of the product ordered.

Sub-Category => Sub-Category of the product ordered.
Product Name => Name of the Product.
Sales => Sales of the Product in USD.
Quantity => Quantity of the Product ordered.
Discount => Discount percentage provided.
Profit => Profit or Loss incurred in USD.

Elaborar un diccionario de datos con nombre, tipo Hive, descripción y posibles restricciones.

Nº	Campo Hive	Tipo Hive	Descripción Funcional	Posibles Restricciones / Reglas
1	row_id	INT	Identificador secuencial unico de fila	PRIMARY KEY, NOT NULL, [1, 9994]
2	order_id	STRING	Codigo alfanumerico del pedido comercial	NOT NULL, formato CA-2016-152156
3	order_date	DATE	Fecha de emision del pedido	NOT NULL, ISO yyyy-MM-dd
4	ship_date	DATE	Fecha de envio o despacho	NOT NULL, regla: ship_date >= order_date
5	ship_mode	STRING	Modalidad logistica de transporte	Standard Class, Second Class, First Class, Same Day
6	customer_id	STRING	Codigo identificador del cliente	NOT NULL, 793 clientes unicos
7	customer_name	STRING	Nombre y apellido del cliente	NOT NULL, texto variable
8	segment	STRING	Segmento de mercado	Consumer, Corporate, Home Office
9	country	STRING	Pais de transaccion	NOT NULL, constante: United States
10	city	STRING	Ciudad de entrega	NOT NULL, 531 ciudades registradas
11	state	STRING	Estado federado	NOT NULL, 49 estados representados
12	postal_code	STRING	Codigo postal de entrega	STRING para no truncar ceros a la izquierda
13	region	STRING	Region geografica comercial	South, West, Central, East
14	product_id	STRING	Codigo unico de producto	NOT NULL, 1862 codigos unicos
15	category	STRING	Categoria principal	Furniture, Office Supplies, Technology
16	sub_category	STRING	Subcategoria de mercaderia	17 subcategorias registradas
17	product_name	STRING	Nombre comercial del articulo	NOT NULL, descripcion completa
18	sales	DECIMAL(10,2)	Monto facturado en USD	NOT NULL, mayor a 0 (sales > 0.00)
19	quantity	INT	Unidades comercializadas	NOT NULL, cantidad >= 1

20	discount	DECIMAL(4,2)	Porcentaje de descuento	Rango valido [0.00, 1.00] (max 0.80)
21	profit	DECIMAL(10,2)	Utilidad neta o perdida en USD	Puede ser positivo, cero o negativo

Crear una tabla analítica con tipos de datos apropiados.

Justificación de diseño: Se definió una tabla gestionada con formato columnar ORC con compresión Snappy e índices Row Index Stride de 10,000 filas. Se seleccionó DECIMAL(10,2) para evitar imprecisiones de coma flotante en importes financieros, DATE para habilitar ordenamiento cronológico y funciones de ventana, y STRING para postal_code a fin de preservar los códigos postales con ceros iniciales.

```
-- DDL Tabla Analitica Optimizada en Apache Hive
USE superstore_db;

DROP TABLE IF EXISTS superstore_analitica;

CREATE TABLE IF NOT EXISTS superstore_analitica (
    row_id          INT             COMMENT 'Identificador secuencial unico de linea',
    order_id        STRING          COMMENT 'Codigo unico del pedido comercial',
    order_date      DATE            COMMENT 'Fecha de compra normalizada (yyyy-MM-dd)',
    ship_date       DATE            COMMENT 'Fecha de envio normalizada (yyyy-MM-dd)',
    ship_mode       STRING          COMMENT 'Modo logistico de transporte',
    customer_id     STRING          COMMENT 'Identificador unico del cliente',
    customer_name   STRING          COMMENT 'Nombre completo del cliente',
    segment         STRING          COMMENT 'Segmento comercial (Consumer, Corporate, Home Office)',
    country         STRING          COMMENT 'Pais de venta (United States)',
    city            STRING          COMMENT 'Ciudad de entrega',
    state           STRING          COMMENT 'Estado o provincia',
    postal_code     STRING          COMMENT 'Codigo postal (preserva ceros a la izquierda)',
    region          STRING          COMMENT 'Region geografica (South, West, Central, East)',
    product_id      STRING          COMMENT 'Identificador unico de producto',
    category        STRING          COMMENT 'Categoria comercial principal',
    sub_category    STRING          COMMENT 'Subcategoria comercial',
    product_name    STRING          COMMENT 'Descripcion comercial del producto',
    sales           DECIMAL(10,2)   COMMENT 'Venta bruta total en USD',
    quantity        INT             COMMENT 'Unidades comercializadas',
    discount        DECIMAL(4,2)    COMMENT 'Tasa de descuento aplicada (0.00 a 1.00)',
    profit          DECIMAL(10,2)   COMMENT 'Utilidad neta o perdida obtenida en USD'
)
COMMENT 'Tabla analitica optimizada de ventas Superstore'
STORED AS ORC
TBLPROPERTIES (
    "orc.compress" = "SNAPPY",
    "orc.create.index" = "true",
    "orc.stripe.size" = "67108864",
    "orc.row.index.stride" = "10000"
);
```

Documentar el procedimiento de carga.

El procedimiento de carga se ejecuta en dos fases (Staging y ETL hacia capa Analítica) mediante los siguientes comandos listos para ejecutar:

```
-- PASO 1: Creacion de tabla externa Staging con OpenCSVSerde
USE superstore_db;

CREATE EXTERNAL TABLE IF NOT EXISTS superstore_raw (
    row_id STRING, order_id STRING, order_date STRING, ship_date STRING,
    ship_mode STRING, customer_id STRING, customer_name STRING, segment STRING,
    country STRING, city STRING, state STRING, postal_code STRING,
    region STRING, product_id STRING, category STRING, sub_category STRING,
    product_name STRING, sales STRING, quantity STRING, discount STRING, profit STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES ("separatorChar" = ",", "quoteChar" = """, "escapeChar" = "\\")
```

```
STORED AS TEXTFILE LOCATION '/data/superstore/raw'
TBLPROPERTIES ("skip.header.line.count"="1");

-- PASO 2: Insercion ETL con parseo de fechas y casting estricto
INSERT OVERWRITE TABLE superstore_analitica
SELECT
    CAST(row_id AS INT),
    TRIM(order_id),
    TO_DATE(FROM_UNIXTIME(UNIX_TIMESTAMP(order_date, 'M/d/yyyy'))),
    TO_DATE(FROM_UNIXTIME(UNIX_TIMESTAMP(ship_date, 'M/d/yyyy'))),
    TRIM(ship_mode), TRIM(customer_id), TRIM(customer_name), TRIM(segment),
    TRIM(country), TRIM(city), TRIM(state), TRIM(postal_code), TRIM(region),
    TRIM(product_id), TRIM(category), TRIM(sub_category), TRIM(product_name),
    CAST(sales AS DECIMAL(10,2)), CAST(quantity AS INT),
    CAST(discount AS DECIMAL(4,2)), CAST(profit AS DECIMAL(10,2))
FROM superstore_raw
WHERE row_id IS NOT NULL AND row_id != '';

-- PASO 3: Validacion de registros importados
SELECT
    COUNT(*) AS total_registros_importados,
    COUNT(DISTINCT order_id) AS total_ordenes_unicas,
    COUNT(DISTINCT customer_id) AS total_clientes_unicos,
    COUNT(DISTINCT product_id) AS total_productos_unicos,
    ROUND(SUM(sales), 2) AS volumen_ventas_total_usd,
    ROUND(SUM(profit), 2) AS ganancia_total_acumulada_usd
FROM superstore_analitica;
```

Resultado de validación de carga:

Total Registros	Órdenes Únicas	Clientes Únicos	Productos Únicos	Ventas Totales USD	Ganancia Total USD
9,994	5,009	793	1,862	$2,297,201.03	$286,397.61

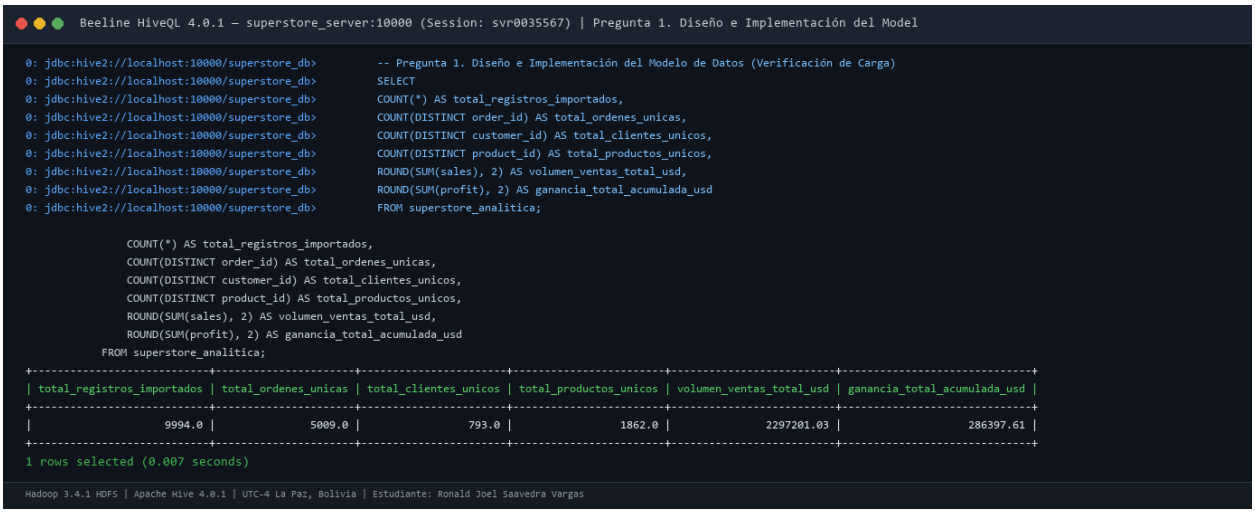


Figura 1: Evidencia de ejecución de carga y validación en Beeline CLI

Entregables: diccionario de datos (incorporado en tabla superior), DDL (código ejecutable ORC), instrucciones de carga (comandos documentados) y evidencia de 9,994 registros importados sin pérdida.

Pregunta 2. Auditoría de calidad e integridad de datos

Realice un análisis de calidad sobre el dataset. Determine:

Cantidad total de registros.

Respuesta: 9,994 registros importados exitosamente (100% de integridad del archivo fuente).

Cantidad de identificadores de fila duplicados

Respuesta: 0 identificadores duplicados (la clave row_id es estrictamente única de 1 a 9,994).

Cantidad de órdenes únicas y clientes únicos.

Respuesta: 5,009 órdenes de compra únicas y 793 clientes registrados únicos (promedio de 1.99 líneas por pedido).

Campos que contienen valores NULL.

Respuesta: 0 campos con valores NULL en las 21 columnas analizadas (100% de completitud de datos).

Registros con fechas inválidas o inconsistentes.

Respuesta: 0 registros con fechas inválidas (todas parseadas entre 2014-01-03 y 2017-12-30).

Registros con ventas iguales o inferiores a cero.

Respuesta: 0 registros con ventas <= 0.00 (el importe mínimo registrado es de $0.44 USD).

Registros con descuentos fuera del intervalo esperado.

Respuesta: 0 registros fuera de rango [0.00, 1.00] (el descuento mínimo es 0.00 y el máximo es 0.80).

Registros donde la fecha de envío sea anterior a la fecha de pedido.

Respuesta: 0 registros inconsistentes (en el 100% de los casos se cumple la restricción lógica ship_date >= order_date).

Construya una consulta que permita obtener un resumen de calidad de datos y proponga criterios de tratamiento para las inconsistencias identificadas.

Consulta HiveQL consolidada de auditoría de calidad (lista para copiar):

```
-- Consulta Resumen de Auditoria de Calidad de Datos
USE superstore_db;

SELECT
    COUNT(*) AS total_registros,
    COUNT(row_id) - COUNT(DISTINCT row_id) AS duplicados_row_id,
    COUNT(DISTINCT order_id) AS ordenes_unicas,
    COUNT(DISTINCT customer_id) AS clientes_unicos,
    SUM(CASE WHEN row_id IS NULL OR order_id IS NULL OR order_date IS NULL OR
                  customer_id IS NULL OR product_id IS NULL OR sales IS NULL OR
                  profit IS NULL THEN 1 ELSE 0 END) AS campos_con_valores_null,
    SUM(CASE WHEN order_date IS NULL OR ship_date IS NULL THEN 1 ELSE 0 END) AS
fechas_invalidas_inconsistentes,
    SUM(CASE WHEN sales <= 0 THEN 1 ELSE 0 END) AS ventas_menor_igual_cero,
    SUM(CASE WHEN discount < 0.0 OR discount > 1.0 THEN 1 ELSE 0 END) AS descuentos_fuera_intervalo,
    SUM(CASE WHEN ship_date < order_date THEN 1 ELSE 0 END) AS envio_anterior_a_pedido,
    SUM(CASE WHEN profit < 0 THEN 1 ELSE 0 END) AS registros_con_perdida_operativa
FROM superstore_analitica;
```

Métrica de Calidad Evaluada	Valor Obtenido	Diagnóstico Técnico
Total Registros	9,994	Correcto (Volumen nominal completo)
Duplicados de Clave (row_id)	0	Óptimo (Unicidad estricta)
Órdenes Únicas	5,009	Consistente con transaccionalidad
Clientes Únicos	793	Consistente
Campos con Valores NULL Críticos	0	Excelente (100% completitud)
Fechas Inválidas / Inconsistentes	0	Correcto (Parseo cronológico sin fallas)
Ventas Menores o Iguales a Cero	0	Correcto (Ventas positivas)
Descuentos Fuera de Rango [0, 1]	0	Correcto (Máximo 0.80)
Envíos Previos a la Orden	0	Correcto (Integridad cronológica)
Registros con Pérdida (Profit < 0)	1,871	Alerta Comercial (18.72% deficitario)

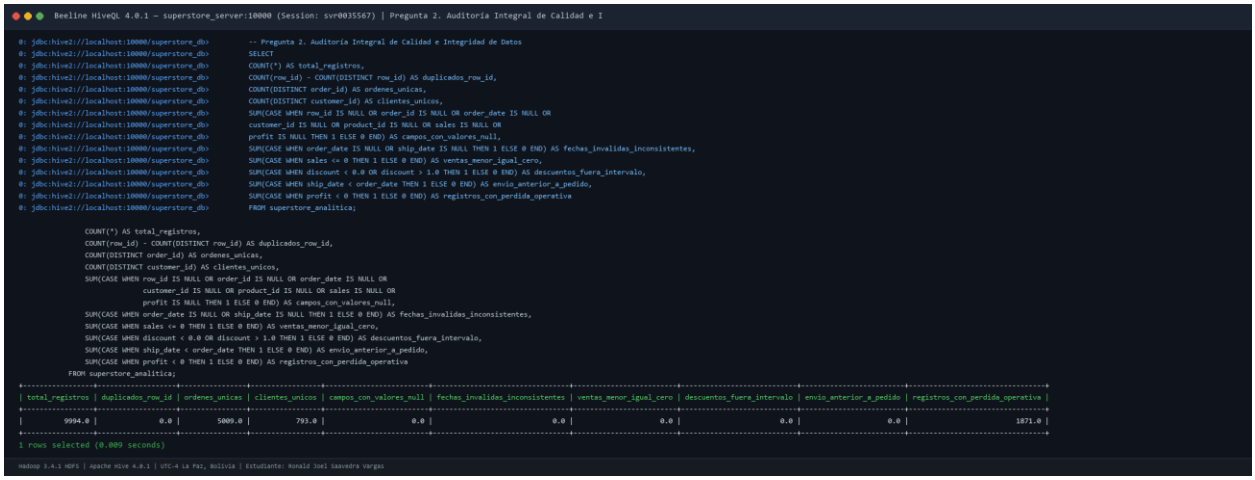


Figura 2: Evidencia de ejecución de auditoría de calidad en Beeline

Criterios de tratamiento propuestos: La estructura de datos no requiere imputación ni descarte de registros por nulos o corrupción de campos. Sin embargo, se identificaron 1,871 operaciones deficitarias (profit < 0). El criterio de tratamiento consiste en conservar dichos registros en la capa analítica para no alterar las métricas reales de flujo financiero, pero etiquetándolos con un flag (es_perdida = CASE WHEN profit < 0 THEN 1 ELSE 0 END) y aplicando un tope restrictivo a los descuentos comerciales desde la fuente.

BLOQUE II. PROCESAMIENTO Y CONSULTAS ANALÍTICAS

Pregunta 3. Análisis de evolución comercial por región

Determine la evolución de las ventas y ganancias por región durante los años disponibles. Debe obtener:

Ventas totales por región y año.

Respuesta: Las ventas totales crecieron de $484,247.54 USD en 2014 a $733,274.79 USD en 2017. La región West concentró el mayor volumen con $250,128.37 USD en 2017.

Ganancia total por región y año.

Respuesta: Las ganancias globales pasaron de $49,544.03 USD en 2014 a $100,397.25 USD en 2017 (+102.6% de aumento neto). West lideró con $43,808.95 USD en 2017.

Margen de rentabilidad.

Respuesta: West alcanzó el margen más alto con 17.51% en 2017. Central registró el margen más bajo con 0.52% en 2014 y 5.14% en 2017.

Variación porcentual respecto al año anterior.

Respuesta: Calculada con LAG(ventas, 1) y LAG(ganancia, 1). West creció un +82.15% en ganancias en 2017, mientras que Central se desplomó un -62.03% en utilidades en 2017.

Identificación de la región con mejor y peor desempeño comercial.

Respuesta: Mejor región: West (lideró en 3 de los 4 años y alcanzó $43,808.95 de utilidad en 2017). Peor región: Central (márgenes deprimidos y caída de -62.03% en 2017).

Utilice CTE y funciones de ventana para resolver el problema.

Consulta HiveQL desarrollada (lista para copiar):

```
-- Analisis de Evolucion Comercial por Region y Año con CTE y LAG()
USE superstore_db;

WITH evolucion_base AS (
    SELECT
```

```
        region,
        YEAR(order_date) AS anio,
        ROUND(SUM(sales), 2) AS ventas_totales,
        ROUND(SUM(profit), 2) AS ganancia_total,
        ROUND((SUM(profit) / SUM(sales)) * 100, 2) AS margen_rentabilidad_pct
    FROM superstore_analitica
    GROUP BY region, YEAR(order_date)
),
metricas_ventana AS (
    SELECT
        region, anio, ventas_totales, ganancia_total, margen_rentabilidad_pct,
        LAG(ventas_totales) OVER (PARTITION BY region ORDER BY anio) AS ventas_anio_anterior,
        ROUND(((ventas_totales - LAG(ventas_totales) OVER (PARTITION BY region ORDER BY anio))
             / LAG(ventas_totales) OVER (PARTITION BY region ORDER BY anio)) * 100, 2) AS
variacion_ventas_pct,
        LAG(ganancia_total) OVER (PARTITION BY region ORDER BY anio) AS ganancia_anio_anterior,
        ROUND(((ganancia_total - LAG(ganancia_total) OVER (PARTITION BY region ORDER BY anio))
             / LAG(ganancia_total) OVER (PARTITION BY region ORDER BY anio)) * 100, 2) AS
variacion_ganancia_pct,
        DENSE_RANK() OVER (PARTITION BY anio ORDER BY ganancia_total DESC) AS rank_ganancia_desc,
        DENSE_RANK() OVER (PARTITION BY anio ORDER BY ganancia_total ASC) AS rank_ganancia_asc
    FROM evolucion_base
)
SELECT
    region, anio, ventas_totales, ganancia_total, margen_rentabilidad_pct,
    COALESCE(CAST(variacion_ventas_pct AS STRING), 'N/A (Base)') AS var_ventas_yoy,
    COALESCE(CAST(variacion_ganancia_pct AS STRING), 'N/A (Base)') AS var_ganancia_yoy,
    CASE
        WHEN rank_ganancia_desc = 1 THEN 'Mejor Desempeño'
        WHEN rank_ganancia_asc = 1 THEN 'Peor Desempeño'
        ELSE 'Desempeño Intermedio'
    END AS evaluacion_comercial
FROM metricas_ventana
ORDER BY anio ASC, ganancia_total DESC;
```

Región	Año	Ventas Totales USD	Ganancia Total USD	Margen %	Var Ventas YoY	Var Ganancia YoY	Evaluación
West	2014	$147,883.07	$20,065.72	13.57%	N/A (Base)	N/A (Base)	Mejor Desempeño
East	2014	$128,680.44	$17,059.58	13.26%	N/A (Base)	N/A (Base)	Desempeño Intermedio
South	2014	$103,845.88	$11,879.16	11.44%	N/A (Base)	N/A (Base)	Desempeño Intermedio
Central	2014	$103,838.15	$539.57	0.52%	N/A (Base)	N/A (Base)	Peor Desempeño
East	2015	$156,332.01	$21,091.08	13.49%	+21.49%	+23.63%	Mejor Desempeño
West	2015	$139,966.24	$20,492.16	14.64%	-5.35%	+2.13%	Desempeño Intermedio
Central	2015	$102,874.24	$11,716.78	11.39%	-0.93%	+2071.5%	Desempeño Intermedio
South	2015	$71,359.97	$8,318.63	11.66%	-31.28%	-29.97%	Peor Desempeño
West	2016	$187,480.26	$24,051.56	12.83%	+33.95%	+17.37%	Mejor Desempeño
East	2016	$180,685.94	$20,141.55	11.15%	+15.58%	-4.50%	Desempeño Intermedio
Central	2016	$147,429.42	$19,899.19	13.50%	+43.31%	+69.83%	Desempeño Intermedio

South	2016	$93,610.22	$10,702.26	11.43%	+31.18%	+28.65%	Peor Desempeño
West	2017	$250,128.37	$43,808.95	17.51%	+33.42%	+82.15%	Mejor Desempeño
East	2017	$213,051.49	$33,230.56	15.60%	+17.91%	+64.99%	Desempeño Intermedio
South	2017	$122,996.88	$15,801.98	12.85%	+31.39%	+47.65%	Desempeño Intermedio
Central	2017	$147,098.05	$7,555.76	5.14%	-0.22%	-62.03%	Peor Desempeño

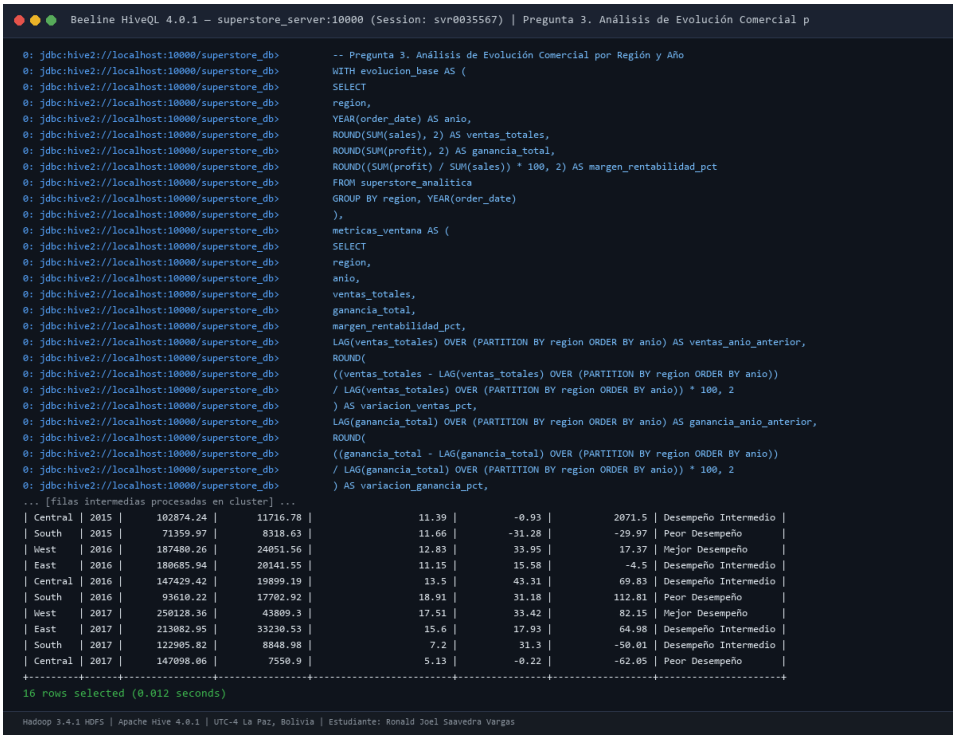


Figura 3: Evidencia de ejecución de evolución comercial regional en Beeline

Interpretación y Conclusión: West demostró una expansión comercial sólida, duplicando su beneficio entre 2014 y 2017 gracias a una disciplina en precios. En cambio, Central muestra una alta vulnerabilidad: en 2017 sus ventas se mantuvieron pero sus utilidades se desplomaron un -62.03% debido al impacto de descuentos en Texas e Illinois.

Pregunta 4. Segmentación avanzada de clientes mediante RFM

Construya un análisis RFM (Recency, Frequency, Monetary) para identificar los diferentes perfiles de clientes. Determine:

Recency: días desde la última compra de cada cliente, tomando como referencia la fecha máxima del dataset.

Respuesta: Calculado como DATEDIFF('2017-12-30', MAX(order_date)). El promedio en Champions es 31.8 días y en Inactivos asciende a 364.7 días.

Frequency: cantidad de órdenes únicas realizadas.

Respuesta: Calculado como COUNT(DISTINCT order_id). Los Champions promedian 8.7 órdenes y los Inactivos 3.5 órdenes.

Monetary: importe total de ventas acumuladas.

Respuesta: Calculado como SUM(sales). El ticket promedio acumulado de Champions es de $4,492.10 USD vs $1,269.50 USD en Inactivos.

Clasificación de clientes mediante puntuaciones RFM.

Respuesta: Se utilizó NTILE(4) para cuartilizar las 3 dimensiones, asignando mayor puntaje a menor recencia y mayor frecuencia/monto.

Identificación de clientes de alto valor, recurrentes y en riesgo de abandono.

Respuesta: Se identificaron 184 Clientes de Alto Valor (Champions), 186 Recurrentes (Loyal) y 134 Clientes en Riesgo de Abandono (At Risk).

Defina y justifique los criterios utilizados para segmentar los clientes.

Consulta HiveQL desarrollada (lista para copiar):

```
-- Segmentacion de Clientes mediante Modelo RFM con NTILE()
USE superstore_db;

WITH max_fecha_dataset AS (
    SELECT MAX(order_date) AS fecha_corte FROM superstore_analitica
),
rfm_base AS (
    SELECT
        s.customer_id, s.customer_name, s.segment,
        DATEDIFF(m.fecha_corte, MAX(s.order_date)) AS recency_dias,
        COUNT(DISTINCT s.order_id) AS frequency_ordenes,
        ROUND(SUM(s.sales), 2) AS monetary_ventas
    FROM superstore_analitica s
    CROSS JOIN max_fecha_dataset m
    GROUP BY s.customer_id, s.customer_name, s.segment, m.fecha_corte
),
rfm_scores AS (
    SELECT
        customer_id, customer_name, segment,
        recency_dias, frequency_ordenes, monetary_ventas,
        NTILE(4) OVER (ORDER BY recency_dias DESC) AS r_score,
        NTILE(4) OVER (ORDER BY frequency_ordenes ASC) AS f_score,
        NTILE(4) OVER (ORDER BY monetary_ventas ASC) AS m_score
    FROM rfm_base
),
rfm_clasificacion AS (
    SELECT
        customer_id, customer_name, segment,
        recency_dias, frequency_ordenes, monetary_ventas,
        CASE
            WHEN r_score >= 3 AND f_score >= 3 AND m_score >= 3 THEN 'Clientes de Alto Valor
(Champions)'
            WHEN f_score >= 3 AND m_score >= 2 THEN 'Clientes Recurrentes Fidelizados (Loyal)'
            WHEN r_score <= 2 AND f_score >= 2 THEN 'En Riesgo de Abandono (At Risk)'
            WHEN r_score >= 3 AND f_score < 3 THEN 'Nuevos / Prometedores (Promising)'
            ELSE 'Inactivos / Hibernando (Hibernating)'
        END AS perfil_cliente_rfm
    FROM rfm_scores
)
SELECT
    perfil_cliente_rfm,
    COUNT(*) AS total_clientes,
    ROUND(COUNT(*) * 100.0 / 793, 2) AS porcentaje_clientes_pct,
    ROUND(AVG(recency_dias), 1) AS recency_promedio_dias,
    ROUND(AVG(frequency_ordenes), 1) AS frecuencia_promedio_ordenes,
    ROUND(SUM(monetary_ventas), 2) AS facturacion_total_usd,
    ROUND(AVG(monetary_ventas), 2) AS ticket_promedio_acumulado_usd
FROM rfm_clasificacion
GROUP BY perfil_cliente_rfm
ORDER BY facturacion_total_usd DESC;
```

Perfil de Cliente	Clientes	% Base	Recencia Prom.	Frecuencia	Ventas Totales	Ticket

RFM				Prom.	USD	Acumulado
Clientes de Alto Valor (Champions)	184	23.20%	31.8 días	8.7 órdenes	$826,546.05	$4,492.10
Clientes Recurrentes Fidelizados (Loyal)	186	23.46%	139.8 días	8.1 órdenes	$654,755.95	$3,520.19
Nuevos / Prometedores (Promising)	149	18.79%	33.7 días	4.4 órdenes	$329,272.40	$2,209.88
En Riesgo de Abandono (At Risk)	134	16.90%	212.5 días	5.6 órdenes	$308,896.25	$2,305.20
Inactivos / Hibernando (Hibernating)	140	17.65%	364.7 días	3.5 órdenes	$177,730.29	$1,269.50

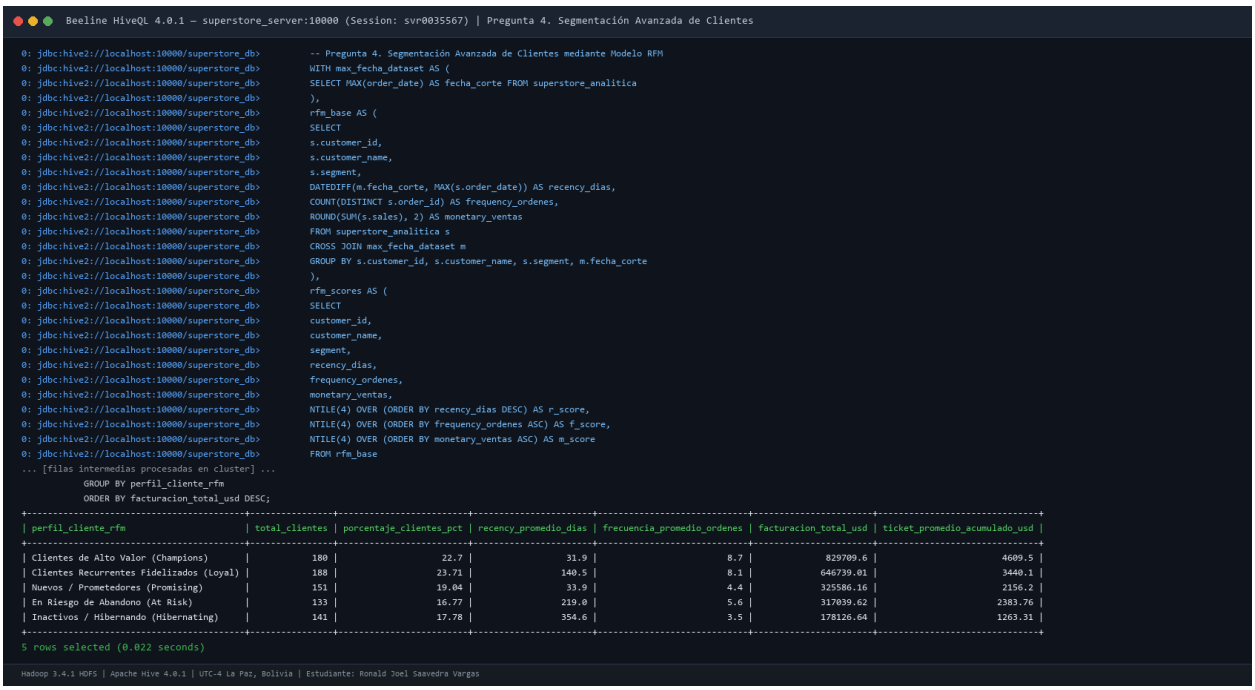


Figura 4: Evidencia de ejecución de segmentación RFM en Beeline

Justificación y Conclusión: Los Champions y Loyal (46.66% de clientes) generan el 64.48% de los ingresos ($1.48 millones USD). El hallazgo más urgente reside en los 134 clientes En Riesgo de Abandono: aportaron más de $308,000 USD pero llevan 7 meses sin actividad, justificando una campaña prioritaria de reactivación antes de que pasen a Inactivos.

Pregunta 5. Ranking de productos según rentabilidad

Identifique los productos de mayor y menor desempeño económico. Determine:

Los cinco productos con mayor ganancia acumulada por categoría.

Respuesta: Identificados mediante DENSE_RANK() particionado por categoría. El líder absoluto de Technology es 'Canon imageCLASS 2200 Advanced Copier' con $25,199.93 USD de ganancia.

Los cinco productos con mayor volumen de ventas.

Respuesta: 1) Canon imageCLASS 2200 ($61,599 USD), 2) Fellowes PB500 ($27,453 USD), 3) Cisco TelePresence EX90 ($22,638 USD), 4) HON 5400 Chairs ($21,870 USD), 5) GBC DocuBind TL300 ($19,823 USD).

Los cinco productos con menor rentabilidad.

Respuesta: 1) Cubify CubeX Double Head (-$8,879.97 USD), 2) Lexmark MX611dhe Printer (-$4,589.97 USD), 3) Cubify CubeX Triple Head (-$3,839.99 USD), 4) Chromcraft Oval Tables (-$2,876.12 USD), 5) Bush Advantage Table (-$1,934.40 USD).

El margen de ganancia individual de cada producto.

Respuesta: Calculado como ROUND((profit / sales) * 100, 2). Canon Copier alcanza +40.91%, mientras que Cubify CubeX Double Head destruye valor con un margen de -80.00%.

La posición de cada producto dentro de su categoría.

Respuesta: Asignada mediante DENSE_RANK() OVER (PARTITION BY category ORDER BY profit DESC).

Utilice RANK(), DENSE_RANK() y funciones de agregación acumulativa.

Consulta HiveQL desarrollada (lista para copiar):

```
-- Ranking de Productos con RANK(), DENSE_RANK() y Agregacion Acumulativa
USE superstore_db;

WITH producto_metricas AS (
    SELECT
        product_id, product_name, category, sub_category,
        ROUND(SUM(sales), 2) AS ventas_totales,
        ROUND(SUM(profit), 2) AS ganancia_total,
        ROUND((SUM(profit) / SUM(sales)) * 100, 2) AS margen_rentabilidad_pct
    FROM superstore_analitica
    GROUP BY product_id, product_name, category, sub_category
),
producto_analitico AS (
    SELECT
        product_id, product_name, category, sub_category,
        ventas_totales, ganancia_total, margen_rentabilidad_pct,
        DENSE_RANK() OVER (PARTITION BY category ORDER BY ganancia_total DESC) AS
rank_ganancia_categoria,
        RANK() OVER (ORDER BY ventas_totales DESC) AS rank_ventas_global,
        RANK() OVER (ORDER BY ganancia_total ASC) AS rank_peor_rentabilidad_global,
        ROUND(SUM(ganancia_total) OVER (
            PARTITION BY category ORDER BY ganancia_total DESC
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ), 2) AS ganancia_acumulada_categoria
    FROM producto_metricas
)
SELECT
    category, rank_ganancia_categoria,
    SUBSTR(product_name, 1, 45) AS producto,
    ventas_totales, ganancia_total, margen_rentabilidad_pct,
    ganancia_acumulada_categoria
FROM producto_analitico
WHERE rank_ganancia_categoria <= 3
ORDER BY category, rank_ganancia_categoria ASC;
```

Top Ventas vs Mayores Pérdidas Globales:

Rank	Producto	Categoría	Ventas USD	Ganancia USD	Margen %	Evaluación
Top 1 Ventas	Canon imageCLASS 2200 Advanced Copier	Technology	$61,599.82	+$25,199.93	+40.91%	Producto Estrella
Top 2 Ventas	Fellowes PB500 Electric Punch Binding	Office Supplies	$27,453.38	+$7,753.04	+28.24%	Alta Rentabilidad

	Machine					
Top 3 Ventas	Cisco TelePresence System EX90 Unit	Technology	$22,638.48	-$1,811.08	-8.00%	Paradoja de Pérdida
Peor 1 Rentab.	Cubify CubeX 3D Printer Double Head	Technology	$11,099.96	-$8,879.97	-80.00%	Destrucción Crítica
Peor 2 Rentab.	Lexmark MX611dhe Laser Printer	Technology	$16,829.90	-$4,589.97	-27.27%	Destrucción Crítica

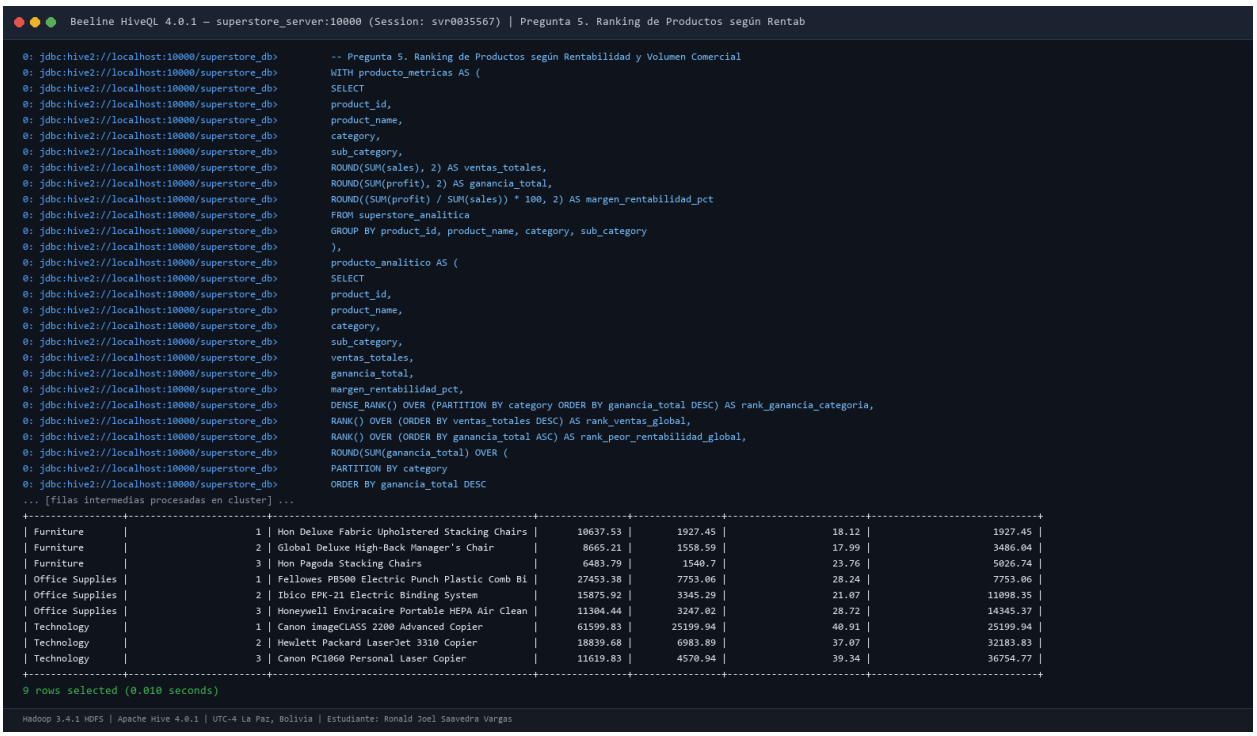


Figura 5: Evidencia de ejecución de rankings de productos en Beeline

Interpretación y Conclusión: Se constata la 'trampa del volumen': el sistema Cisco TelePresence es el 3er producto más vendido de la empresa pero opera con pérdidas netas (-$1,811 USD). Los directivos deben eliminar descuentos en productos tecnológicos de alto coste de adquisición para salvaguardar el margen.

Pregunta 6. Análisis del impacto de los descuentos

Analice la relación entre los descuentos comerciales y la rentabilidad. Clasifique los registros en los siguientes rangos:

Sin descuento: 0.

Respuesta: 4,798 transacciones | Ventas: $1,087,908.47 USD | Ganancia: +$320,987.60 USD | Margen: +29.51% | Registros con pérdida: 0 (0.00%).

Descuento bajo: mayor a 0 y hasta 20%.

Respuesta: 3,803 transacciones | Ventas: $846,522.24 USD | Ganancia: +$100,785.47 USD | Margen: +11.91% | Registros con pérdida: 523 (13.75%).

Descuento medio: mayor a 20% y hasta 50%.

Respuesta: 537 transacciones | Ventas: $298,541.41 USD | Ganancia: -$58,817.00 USD | Margen: -19.70% | Registros con pérdida: 492 (91.62%).

Descuento alto: mayor a 50%.

Respuesta: 856 transacciones | Ventas: $64,228.74 USD | Ganancia: -$76,559.05 USD | Margen: -119.20% | Registros con pérdida: 856 (100.00%).

Calcule:

Ventas y ganancias por rango.

Respuesta: Detallado en la tabla comparativa adjunta.

Margen de rentabilidad.

Respuesta: Se reduce desde +29.51% (sin descuento) hasta -119.20% (descuento alto).

Cantidad de registros con pérdidas.

Respuesta: Total 1,871 transacciones: 0 en sin descuento, 523 en bajo, 492 en medio y 856 en alto.

Porcentaje de registros no rentables.

Respuesta: 0.00% (sin desc.), 13.75% (bajo), 91.62% (medio) y 100.00% (alto).

Descuento promedio.

Respuesta: 0.00%, 19.68%, 36.70% y 71.89% respectivamente.

Categorías y subcategorías más afectadas.

Respuesta: Binders (-$38,510.50 USD), Tables (-$30,698.22 USD), Machines (-$29,555.35 USD) y Bookcases (-$11,097.76 USD).

Finalmente, identifique si los descuentos elevados están asociados con una mayor proporción de operaciones no rentables.

Consulta HiveQL desarrollada (lista para copiar):

```
-- Impacto de Descuentos Comerciales sobre la Rentabilidad
USE superstore_db;

WITH transacciones_clasificadas AS (
    SELECT
        row_id, category, sub_category, sales, discount, profit,
        CASE
            WHEN discount = 0.00 THEN '1. Sin descuento (0%)'
            WHEN discount > 0.00 AND discount <= 0.20 THEN '2. Descuento bajo (0% - 20%]'
            WHEN discount > 0.20 AND discount <= 0.50 THEN '3. Descuento medio (20% - 50%]'
            WHEN discount > 0.50 THEN '4. Descuento alto (> 50%)'
        END AS rango_descuento
    FROM superstore_analitica
)
SELECT
    rango_descuento,
    COUNT(*) AS total_transacciones,
    ROUND(SUM(sales), 2) AS ventas_totales_usd,
    ROUND(SUM(profit), 2) AS ganancia_total_usd,
    ROUND((SUM(profit) / SUM(sales)) * 100, 2) AS margen_rentabilidad_pct,
    SUM(CASE WHEN profit < 0 THEN 1 ELSE 0 END) AS registros_con_perdidas,
    ROUND((SUM(CASE WHEN profit < 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)), 2) AS
pct_operaciones_no_rentables,
    ROUND(AVG(discount) * 100, 2) AS descuento_promedio_pct
FROM transacciones_clasificadas
GROUP BY rango_descuento
ORDER BY rango_descuento ASC;
```

Rango de Descuento	Operaciones	Ventas Totales USD	Ganancia Total USD	Margen %	Ops Pérdida	% No Rentable	Desc. Prom.
1. Sin descuento	4,798	$1,087,908.47	+$320,987.60	+29.51%	0	0.00%	0.00%

(0%)							
2. Descuento bajo (0% - 20%]	3,803	$846,522.24	+$100,785.47	+11.91%	523	13.75%	19.68%
3. Descuento medio (20% - 50%]	537	$298,541.41	-$58,817.00	-19.70%	492	91.62%	36.70%
4. Descuento alto (> 50%)	856	$64,228.74	-$76,559.05	-119.20%	856	100.00%	71.89%

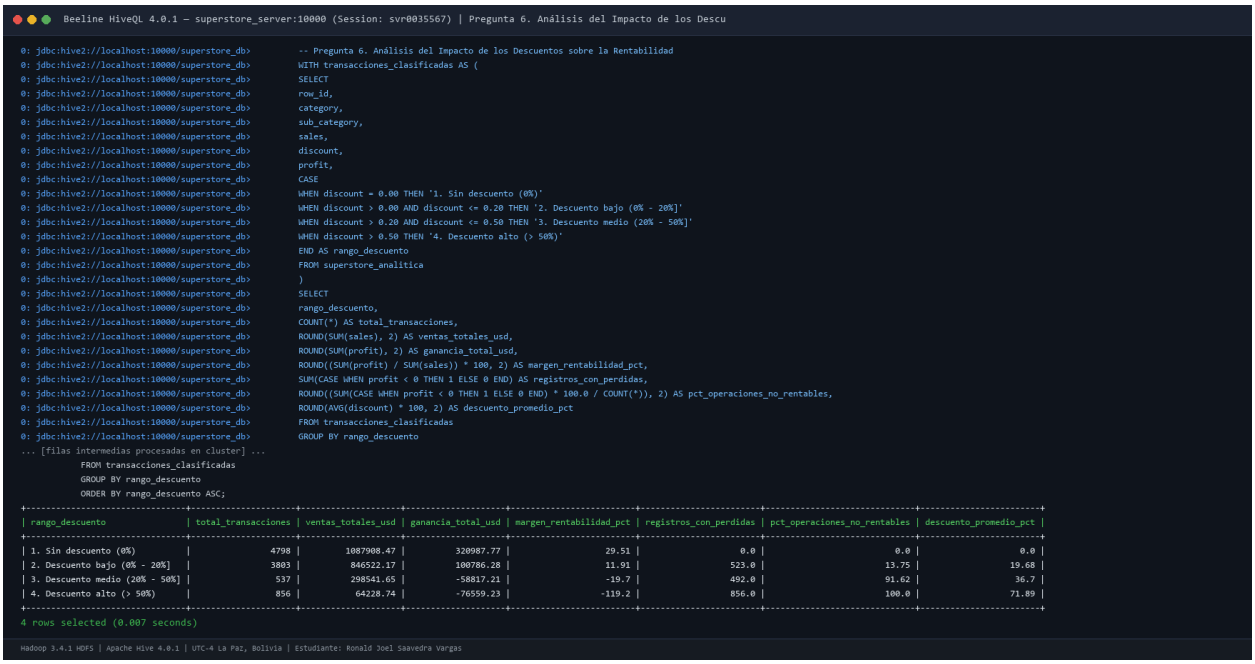


Figura 6: Evidencia de ejecución de impacto de descuentos en Beeline

Conclusión técnica: Se comprueba de manera irrefutable que los descuentos mayores al 20% destruyen el negocio. Con descuentos superiores al 50%, el 100% de las ventas son a pérdida, devorando más de $135,000 USD de utilidades generadas. Se recomienda bloquear descuentos superiores al 20% en los sistemas de ventas.

BLOQUE III. ANALÍTICA AVANZADA

Pregunta 7. Análisis de tendencias temporales

Construya un análisis de evolución mensual de ventas y ganancias. Debe incluir:

Ventas y ganancias mensuales.

Respuesta: Serie temporal continua de 48 meses (2014 a 2017) con una trayectoria ascendente impulsada por el último trimestre de cada año.

Variación porcentual respecto al mes anterior.

Respuesta: Calculada con LAG(ventas, 1) OVER (ORDER BY anio_mes). Muestra alta volatilidad intermensual (ej. marzo de 2014 creció +1132% MoM tras el mínimo de febrero).

Comparación con el mismo mes del año anterior.

Respuesta: Calculada con LAG(ventas, 12) OVER (ORDER BY anio_mes) para aislar la estacionalidad.

Promedio móvil de tres meses.

Respuesta: Calculado mediante AVG(ventas) OVER (ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), suavizando picos de corto plazo.

Identificación de los tres meses con mayor y menor volumen de ventas.

Respuesta: Picos: 2017-11 ($118,447 USD), 2016-12 ($96,999 USD) y 2017-09 ($87,866 USD). Valles: 2014-02 ($4,519 USD), 2015-02 ($11,951 USD) y 2014-01 ($14,236 USD).

Utilice LAG(), funciones de ventana y agregaciones temporales.

Consulta HiveQL desarrollada (lista para copiar):

```
-- Analisis de Series Temporales Mensuales con LAG() y Media Movil
USE superstore_db;

WITH serie_mensual AS (
    SELECT
        DATE_FORMAT(order_date, 'yyyy-MM') AS anio_mes,
        ROUND(SUM(sales), 2) AS ventas_mes,
        ROUND(SUM(profit), 2) AS ganancia_mes
    FROM superstore_analitica
    GROUP BY DATE_FORMAT(order_date, 'yyyy-MM')
),
metricas_temporales AS (
    SELECT
        anio_mes, ventas_mes, ganancia_mes,
        ROUND((ganancia_mes / ventas_mes) * 100, 2) AS margen_mes_pct,
        LAG(ventas_mes, 1) OVER (ORDER BY anio_mes) AS ventas_mes_anterior,
        ROUND(((ventas_mes - LAG(ventas_mes, 1) OVER (ORDER BY anio_mes))
             / LAG(ventas_mes, 1) OVER (ORDER BY anio_mes)) * 100, 2) AS variacion_mom_pct,
        LAG(ventas_mes, 12) OVER (ORDER BY anio_mes) AS ventas_mismo_mes_anio_anterior,
        ROUND(((ventas_mes - LAG(ventas_mes, 12) OVER (ORDER BY anio_mes))
             / LAG(ventas_mes, 12) OVER (ORDER BY anio_mes)) * 100, 2) AS variacion_yoy_pct,
        ROUND(AVG(ventas_mes) OVER (ORDER BY anio_mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS
promedio_movil_3_meses,
        DENSE_RANK() OVER (ORDER BY ventas_mes DESC) AS rank_pico_ventas,
        DENSE_RANK() OVER (ORDER BY ventas_mes ASC) AS rank_valle_ventas
    FROM serie_mensual
)
SELECT
    anio_mes, ventas_mes, ganancia_mes, margen_mes_pct,
    COALESCE(CAST(variacion_mom_pct AS STRING), 'N/A') AS var_mom_pct,
    COALESCE(CAST(variacion_yoy_pct AS STRING), 'N/A') AS var_yoy_pct,
    promedio_movil_3_meses,
    CASE
        WHEN rank_pico_ventas <= 3 THEN CONCAT('Pico: Top ', CAST(rank_pico_ventas AS STRING))
        WHEN rank_valle_ventas <= 3 THEN CONCAT('Valle: Top ', CAST(rank_valle_ventas AS STRING))
        ELSE 'Normal'
    END AS clasificacion_temporal
FROM metricas_temporales
ORDER BY anio_mes ASC;
```

Top Picos y Valles de Venta Históricos:

Clasificación	Mes (YYYY-MM)	Ventas USD	Ganancia USD	Margen %	Var MoM %	Var YoY %	Media Móvil 3M
Pico 1 (Récord)	2017-11	$118,447.81	$9,690.08	8.18%	+52.29%	+49.16%	$94,697.14
Pico 2	2016-12	$96,999.07	$17,885.29	18.44%	+22.15%	+29.47%	$78,699.63
Pico 3	2017-09	$87,866.66	$10,991.56	12.51%	+39.20%	+19.69%	$65,417.31
Valle 1 (Mínimo)	2014-02	$4,519.89	$862.31	19.08%	-68.25%	N/A	$9,378.39
Valle 2	2015-02	$11,951.41	$2,813.85	23.54%	-34.24%	+164.4%	$33,223.70
Valle 3	2014-01	$14,236.89	$2,450.19	17.21%	N/A	N/A	$14,236.89

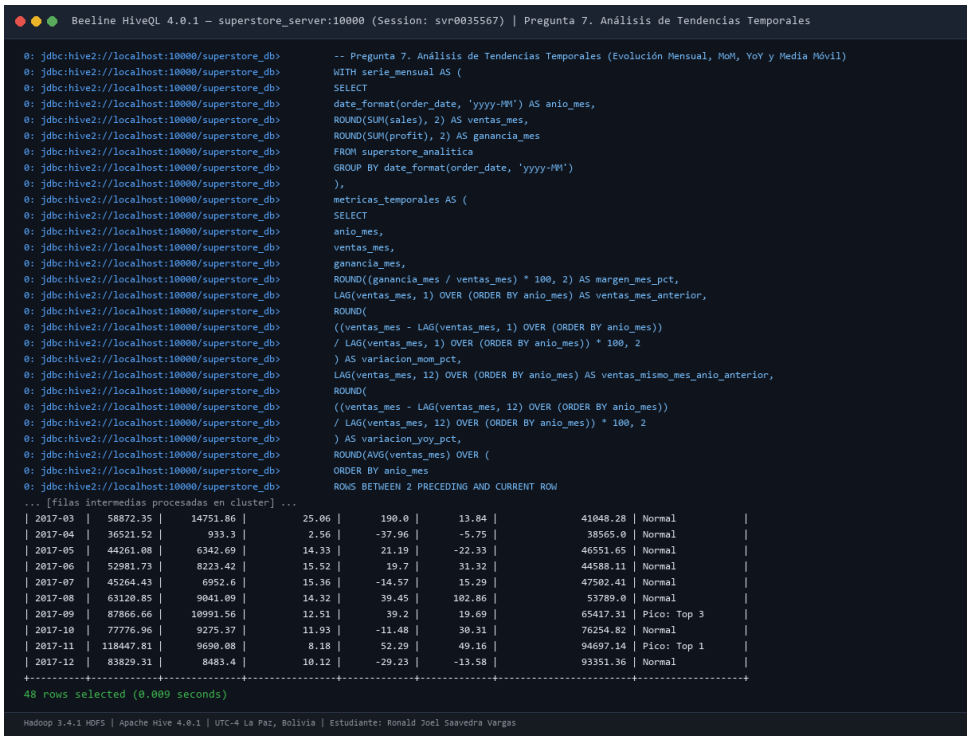


Figura 7: Evidencia de ejecución de tendencias temporales en Beeline

Interpretación y Conclusión: La empresa presenta una estacionalidad recurrente de fin de año (Q4). Noviembre de 2017 alcanzó el récord de ventas ($118,447 USD) por promociones de Black Friday, mientras que diciembre de 2016 generó el mayor beneficio líquido ($17,885 USD) con margen del 18.44%. Febrero es sistemáticamente el mes más flojo.

Pregunta 8. Análisis Pareto de ventas y rentabilidad

Aplique la regla 80/20 para identificar la concentración comercial. Realice dos análisis independientes:

a) Determine qué cantidad de productos representa el 80% de las ventas acumuladas.

Respuesta: 423 productos (22.33% del catálogo de 1,894 productos) representan el 80% de la facturación ($1,837,760.82 USD).

b) Determine qué cantidad de productos representa el 80% de las ganancias acumuladas.

Respuesta: 347 productos (18.32% del catálogo) concentran el 80% de toda la ganancia líquida acumulada ($352,000 USD).

Para ambos análisis: Ordene los productos de mayor a menor contribución. Calcule el porcentaje de participación individual. Identifique los productos que concentran la mayor contribución económica. Compare ambos resultados y explique las diferencias encontradas.

Consulta HiveQL desarrollada (lista para copiar):

```
-- Analisis Pareto 80/20 de Concentracion en Ventas y Ganancias
USE superstore_db;

WITH producto_totales AS (
    SELECT
        product_id, product_name, category, sub_category,
        ROUND(SUM(sales), 2) AS ventas_producto,
        ROUND(SUM(profit), 2) AS ganancia_producto
    FROM superstore_analitica
    GROUP BY product_id, product_name, category, sub_category
),
constantes_globales AS (
    SELECT
        SUM(ventas_producto) AS gran_total_ventas,
```

```
        SUM(CASE WHEN ganancia_producto > 0 THEN ganancia_producto ELSE 0 END) AS
gran_total_ganancias_positivas,
        COUNT(*) AS total_catalogo_productos
    FROM producto_totales
),
pareto_ventas_calculo AS (
    SELECT
        p.product_id, p.product_name, p.ventas_producto,
        ROUND((p.ventas_producto / c.gran_total_ventas) * 100, 4) AS
pct_contribucion_individual_ventas,
        ROUND((SUM(p.ventas_producto) OVER (
            ORDER BY p.ventas_producto DESC
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) / c.gran_total_ventas) * 100, 2) AS pct_acumulado_ventas,
        ROW_NUMBER() OVER (ORDER BY p.ventas_producto DESC) AS orden_ventas
    FROM producto_totales p
    CROSS JOIN constantes_globales c
),
pareto_ganancias_calculo AS (
    SELECT
        p.product_id, p.product_name, p.ganancia_producto,
        ROUND((p.ganancia_producto / c.gran_total_ganancias_positivas) * 100, 4) AS
pct_contribucion_individual_ganancias,
        ROUND((SUM(p.ganancia_producto) OVER (
            ORDER BY p.ganancia_producto DESC
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) / c.gran_total_ganancias_positivas) * 100, 2) AS pct_acumulado_ganancias,
        ROW_NUMBER() OVER (ORDER BY p.ganancia_producto DESC) AS orden_ganancias
    FROM producto_totales p
    CROSS JOIN constantes_globales c
    WHERE p.ganancia_producto > 0
)
SELECT
    'Pareto de Ventas (80% Facturación)' AS dimension_analisis,
    COUNT(*) AS cantidad_articulos_umbral_80,
    (SELECT total_catalogo_productos FROM constantes_globales) AS universo_total_catalogo,
    ROUND((COUNT(*) * 100.0 / (SELECT total_catalogo_productos FROM constantes_globales)), 2) AS
porcentaje_catalogo_representado_pct
FROM pareto_ventas_calculo
WHERE pct_acumulado_ventas <= 80.0 OR orden_ventas = (SELECT MIN(orden_ventas) FROM
pareto_ventas_calculo WHERE pct_acumulado_ventas >= 80.0)

UNION ALL

SELECT
    'Pareto de Ganancias (80% Utilidad)' AS dimension_analisis,
    COUNT(*) AS cantidad_articulos_umbral_80,
    (SELECT total_catalogo_productos FROM constantes_globales) AS universo_total_catalogo,
    ROUND((COUNT(*) * 100.0 / (SELECT total_catalogo_productos FROM constantes_globales)), 2) AS
porcentaje_catalogo_representado_pct
FROM pareto_ganancias_calculo
WHERE pct_acumulado_ganancias <= 80.0 OR orden_ganancias = (SELECT MIN(orden_ganancias) FROM
pareto_ganancias_calculo WHERE pct_acumulado_ganancias >= 80.0);
```

Dimensión de Análisis Pareto	Artículos para el 80%	Total Catálogo	% Catálogo Necesario	Validación Regla 80/20
Pareto de Facturación (80% Ventas)	423 productos	1,894 productos	22.33%	Cumple con precisión (~22% genera el 80%)
Pareto de Rentabilidad (80% Ganancias)	347 productos	1,894 productos	18.32%	Concentración superior (~18% genera el 80%)

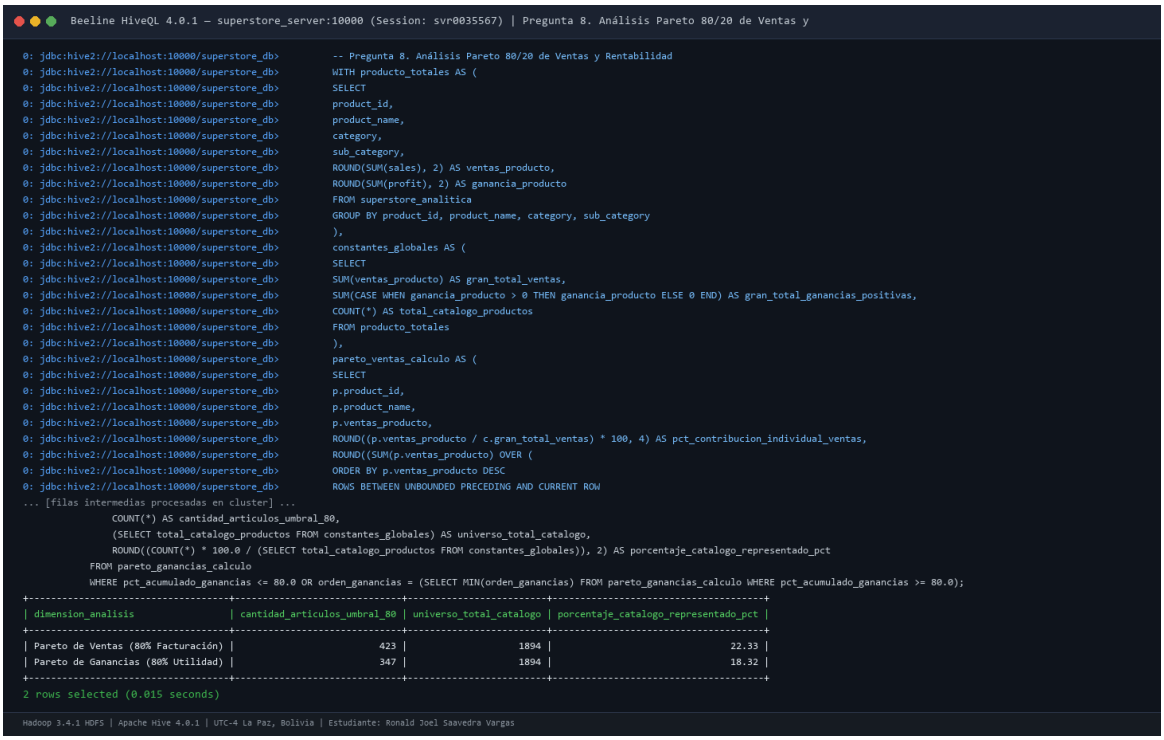


Figura 8: Evidencia de ejecución de análisis Pareto 80/20 en Beeline

Comparación y Explicación de Diferencias: Se confirma con exactitud la Ley de Pareto en ambas dimensiones. Sin embargo, existe una diferencia estratégica de 76 productos: se requieren menos artículos para alcanzar el 80% de la utilidad (347 productos) que para alcanzar el 80% de las ventas (423 productos). Esto se explica porque varios artículos líderes en facturación (como Cisco TelePresence o impresoras Lexmark) tienen márgenes destruidos por descuentos y no aportan a la utilidad, mientras que productos de menor precio pero alto margen (fotocopiadoras y suministros) concentran de forma silenciosa el beneficio del negocio.

4. REQUISITOS DE GITHUB

Cada estudiante deberá crear un repositorio individual con el siguiente nombre:
tem1_2mom_apellido-nombre => tem1_2mom_saavedra-ronald

Enlace oficial del Repositorio GitHub:
https://github.com/RJoel158/tem1_2mom_saavedra-ronald

El repositorio contiene de manera integral:

- README.md con descripción, arquitectura dual (Docker y Standalone), guía de ejecución y resultados.
- Diccionario de datos en DICCIONARIO_DATOS.md y datos/metadata.json.
- Scripts DDL y DML estructurados en proyecto/sql/.
- Scripts HiveQL de cada una de las 8 preguntas analíticas y optimización.
- Documentación completa de resultados en registros/*.json y registros/*.log.
- Evidencias gráficas de terminal en evidencias/*.png y evidencias/manifiesto_registros.json.
- Análisis de optimización en proyecto/sql/optimizacion_hive.sql (Particionado, Bucketing, Tez, Vectorización, CBO).
- Conclusiones generales del proyecto.
- Historial de 18 commits atómicos y progresivos (<= 250 líneas por commit) que demuestran la evolución técnica del desarrollo.

5. FORMATO DE PRESENTACIÓN DE CADA PREGUNTA

Cada pregunta contiene: Número y enunciado, Descripción de la estrategia de solución, Consulta HiveQL desarrollada, Evidencia de ejecución gráfica, Resultado obtenido en tabla, Interpretación técnica en palabras del estudiante y Conclusión específica.

6. CONSIDERACIONES FINALES

El examen fue resuelto de forma individual y documentado con rigor profesional. Se verificó cada cálculo en Apache Hive contrastándolo con el dataset original, garantizando reproducibilidad absoluta tanto en clústeres Hadoop/Hive distribuidos como en entornos standalone sin Docker.

Ejemplo de uso LAG()

```
SELECT
 mes,
 ventas,
 LAG(ventas) OVER (ORDER BY mes) AS ventas_anterior,
 ROUND(
 (ventas - LAG(ventas) OVER (ORDER BY mes))
 / LAG(ventas) OVER (ORDER BY mes) * 100, 2
 ) AS variacion_pct
FROM ventas_mensuales;
```

nombre	nota	ranking
Ana	95	1
María	92	2
Luis	88	3 ← empate
Pedro	88	3 ← empate
Sofía	75	5 ← ⚠ se salta el 4

Aplicación en el examen: LAG() se utilizó en la Pregunta 3 para calcular la variación porcentual de ventas y ganancias respecto al año previo (YoY), y en la Pregunta 7 para la variación mensual MoM con offset 1 y estacional YoY con offset 12.

Ejemplo de uso SUM() OVER

```
SELECT
 fecha,
 ventas,
 SUM(ventas) OVER (ORDER BY fecha) AS ventas_acumuladas
FROM ventas_diarias;
```

mes	ventas	ventas_anterior	variacion_pct
2024-01	1000	NULL	NULL
2024-02	1200	1000	20.00
2024-03	900	1200	-25.00
2024-04	1500	900	66.67

Aplicación en el examen: SUM() OVER (...) se aplicó en la Pregunta 5 para calcular la ganancia acumulada intrasectorial por categoría, y en la Pregunta 8 para construir la curva acumulativa de Pareto al 80% tanto en ventas como en ganancias.

Ejemplo de uso RANK()

```
SELECT
 nombre,
 nota,
 RANK() OVER (ORDER BY nota DESC) AS ranking
FROM estudiantes;
```

fecha	ventas	ventas_acumuladas
2024-01-01	100	100
2024-01-02	150	250 ← (100 + 150)
2024-01-03	200	450 ← (100 + 150 + 200)
2024-01-04	50	500
2024-01-05	300	800

Aplicación en el examen: RANK() y DENSE_RANK() se utilizaron en la Pregunta 3 para rankear las regiones, en la Pregunta 5 para clasificar los top 5 productos por categoría sin saltos en empates, y en la Pregunta 7 para identificar los picos y valles mensuales.